# Primetrade Assignment Interview Explanation

## Project Summary

I built a full-stack assignment centered on a scalable REST API with authentication, role-based access, and a small React frontend to demonstrate the backend. The backend was the primary focus, and the frontend was designed to validate and consume the APIs cleanly.

## How I Approached It

1. I first set up the backend with Express and MongoDB using a modular MVC structure.
2. I created user authentication with registration and login.
3. I added password hashing using bcryptjs and JWT-based authentication.
4. I introduced roles, mainly user and admin, so access rules could differ by route and by data ownership.
5. I created a secondary entity, tasks, and implemented full CRUD APIs for it.
6. I versioned the APIs under /api/v1.
7. I added Swagger docs and a Postman collection so the API could be tested easily.
8. Then I built a React frontend for register, login, protected dashboard access, and task CRUD.
9. After that, I tightened validation, error handling, and frontend UX.

## Backend Story

- I used Express for routing and middleware.
- I used Mongoose models for User and Task.
- In auth, register creates users, login verifies credentials, passwords are hashed before save, and JWT is returned after successful auth.
- Protected routes require a valid bearer token.
- Admin-only routes use role checks, while regular users can only access their own tasks.
- For tasks, I implemented create, list, get by id, update, and delete.
- I added filtering, sorting, and pagination to task APIs.
- I added centralized validation middleware and centralized error handling to keep responses consistent.
- I documented the routes using Swagger and prepared a Postman collection.

## Frontend Story

- I used React with Vite.
- I created pages for login, register, and dashboard.
- I added route protection so unauthenticated users cannot open the dashboard.
- I connected the frontend to the backend using Axios.
- I centralized API calls and token attachment using a shared API client.
- I improved UX by showing both error and success messages for auth and CRUD actions.

- I added basic client-side sanitization and validation before sending data.

## Important Problem I Found and Fixed

One important issue was that the frontend dashboard always called the task stats endpoint, but I later made that endpoint admin-only for cleaner RBAC. That broke the dashboard for normal users. I fixed it by keeping backend stats for admins and deriving task stats from the user?s own task list for regular users.

## Database Behavior

- The app is configured for MongoDB using mongodb://localhost:27017/primetrade_tasks.
- I also added a fallback using mongodb-memory-server.
- So if local MongoDB is unavailable, the backend still runs using an in-memory temporary database.
- When I tested it on this machine, local MongoDB was not installed or running, so the backend automatically fell back to in-memory MongoDB and auth plus CRUD still worked.

## Security Decisions

- Password hashing with bcrypt.
- JWT-protected routes.
- Role-based access control.
- Request validation with express-validator.
- Centralized error handling.
- Basic frontend input sanitization.
- Shared token handling in the frontend.

## Scalability Decisions

- Modular folder structure.
- Route, controller, and model separation.
- Reusable middleware for auth, validation, and errors.
- API versioning.
- Caching support in the backend.
- Docker compose present for database setup.
- A separate scalability note included in the repository.

## Short Interview Answer

I built a REST API with Express and MongoDB for user authentication and task management. I implemented registration, login, password hashing, JWT authentication, and role-based access for user and admin. Then I added task CRUD with filtering, sorting, pagination, validation, error handling, Swagger docs, and a Postman collection. On top of that, I built a React frontend with login, register, a protected dashboard, and task CRUD to demonstrate the APIs. I also fixed integration issues between backend RBAC and frontend behavior, and I added a fallback in-memory MongoDB setup so the app still works even when local MongoDB is unavailable.

## If They Ask About Challenges

- Aligning role-based backend rules with frontend behavior.
- Improving validation and centralized error handling.
- Handling database availability by using in-memory fallback for development and demo.
- Making sure the frontend reflected real API success and failure states.

## If They Ask What You Would Improve Next

- Move JWT handling to httpOnly cookies.
- Add automated tests.
- Add refresh token flow.
- Use Redis instead of in-process cache.
- Add deployment config and CI/CD.
- Use a persistent MongoDB instance instead of in-memory fallback for production.